Machine Learning Basics - Study Notes

These notes cover the fundamentals of machine learning, from core concepts to deployment. They are intended as practice material for building a question-answering chatbot.

1. What is Machine Learning?

Machine Learning (ML) is a branch of Artificial Intelligence in which computers learn patterns from data instead of being given fixed rules by a programmer. In traditional programming, a developer writes the rules and the computer applies them to data to produce answers. In machine learning, the developer provides data and the correct answers, and the computer discovers the rules by itself.

A simple example is email spam detection. Instead of writing hundreds of rules about suspicious words, we show the algorithm thousands of emails that are already labeled as spam or not spam. The algorithm studies these examples and learns which patterns are linked to spam. After training, it can classify new emails it has never seen.

Machine learning is used in many places: product recommendations on shopping sites, voice assistants, face unlock on phones, fraud detection in banking, medical image analysis, language translation, and self-driving cars. The three main ingredients of any ML system are data, a model, and an algorithm that trains the model. The quality of the data usually matters more than the choice of algorithm.

Important terms: a feature is an input variable used to make a prediction, such as the age or salary of a customer. A label or target is the answer we want to predict, such as whether the customer will leave. A model is the mathematical function that maps features to predictions. Training is the process of adjusting the model so its predictions match the labels as closely as possible.

2. Types of Machine Learning

Supervised learning uses labeled data. Each training example has input features and a known correct output. The model learns the relationship between them. Supervised learning has two main problem types. Classification predicts a category, such as spam or not spam. Regression predicts a number, such as the price of a house.

Unsupervised learning uses data without labels. The goal is to find hidden structure. Clustering groups similar items together, for example grouping customers with similar buying habits. Dimensionality reduction compresses many features into fewer ones while keeping the important information. Principal Component Analysis (PCA) is a popular technique for this.

Reinforcement learning is learning by trial and error. An agent takes actions in an environment and receives rewards or penalties. Over time it learns a strategy, called a policy, that maximizes total reward. This approach is used to train game-playing programs and robots.

Semi-supervised learning mixes a small amount of labeled data with a large amount of unlabeled data. Self-supervised learning creates labels from the data itself, for example by hiding a word in a sentence and asking the model to predict it. Modern large language models are trained with self-supervised learning on huge amounts of text.

3. Data Preprocessing

Real-world data is messy. Before training a model, we must clean and prepare it. This stage is called data preprocessing, and it often takes more than half of the total time in an ML project.

Handling missing values: some rows may have empty cells. Common solutions are removing the rows, filling numeric gaps with the mean or median, and filling category gaps with the most frequent value. The right choice depends on how much data is missing and why.

Encoding categorical variables: models work with numbers, not text labels. One-hot encoding turns a column like color with values red, blue, green into three separate columns containing 0 or 1. Label encoding assigns an integer to each category and is suitable when categories have a natural order.

Feature scaling: features with very different ranges can confuse some algorithms. Standardization rescales a feature to have mean 0 and standard deviation 1. Min-max scaling rescales values to the range 0 to 1. Scaling is important for algorithms based on distance or gradients, such as K-Nearest Neighbors and neural networks.

Train-test split: we always keep a portion of the data, commonly 20 percent, hidden from the model during training. This test set is used at the end to measure how well the model performs on new data. A validation set is also used during development to tune settings without touching the test set.

4. Linear and Logistic Regression

Linear regression is one of the simplest ML algorithms. It predicts a continuous number by fitting a straight line, or a flat plane in higher dimensions, through the data. The model is written as y = w1*x1 + w2*x2 + ... + b, where w values are weights and b is the bias. Training finds the weights that minimize the mean squared error between predictions and true values.

Gradient descent is the common optimization method. It starts with random weights, measures the error, and repeatedly nudges the weights in the direction that reduces the error. The size of each nudge is controlled by the learning rate. If the learning rate is too large, training may overshoot and fail to converge. If it is too small, training becomes very slow.

Logistic regression, despite its name, is used for classification. It applies the sigmoid function to a linear combination of features, which squeezes the output into a probability between 0 and 1. If the probability is above a threshold, usually 0.5, the model predicts the positive class. It is widely used for yes-or-no problems such as predicting loan default or customer churn.

Strengths of these models are speed, simplicity, and interpretability: you can read the weights to see which features matter most. Their weakness is that they assume a simple relationship between features and the target, so they may perform poorly on complex patterns.

5. Decision Trees and Random Forests

A decision tree makes predictions by asking a series of yes-or-no questions about the features, like a flowchart. At each step the tree chooses the question that best separates the data. Measures such as Gini impurity and entropy are used to decide how good a split is. The final answer is found at a leaf node.

Decision trees are easy to understand and visualize, and they need little data preparation. However, a deep tree can memorize the training data and overfit. To control this, we limit the maximum depth, require a minimum number of samples per leaf, or prune the tree after training.

A random forest combines many decision trees. Each tree is trained on a random sample of the rows and a random subset of the features. For classification, the forest takes a majority vote of all trees. For regression, it averages their predictions. Because the trees make different mistakes, combining them reduces variance and gives much better accuracy than a single tree.

Gradient boosting methods such as XGBoost, LightGBM, and CatBoost build trees one after another, where each new tree tries to correct the errors of the previous ones. These methods often win competitions on tabular data and are very popular in industry for tasks like churn prediction, credit scoring, and sales forecasting.

6. Model Evaluation Metrics

Choosing the right metric is as important as choosing the right model. For regression problems, common metrics are Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R-squared, which tells how much of the variation in the target is explained by the model.

For classification, accuracy is the fraction of correct predictions. Accuracy can be misleading when classes are imbalanced. For example, if only 1 percent of transactions are fraud, a model that always predicts not fraud gets 99 percent accuracy but is useless.

A confusion matrix counts true positives, true negatives, false positives, and false negatives. From it we calculate precision, recall, and the F1 score. Precision is the share of predicted positives that are truly positive. Recall is the share of actual positives that the model found. The F1 score is the harmonic mean of precision and recall and gives one balanced number.

The ROC curve plots the true positive rate against the false positive rate at different thresholds, and the area under it, called AUC, summarizes performance across all thresholds. In fraud detection or disease screening, recall is often more important than precision because missing a real case is costly.

Cross-validation gives a more reliable estimate of performance. In k-fold cross-validation, the data is split into k parts. The model is trained k times, each time using a different part as the validation set, and the scores are averaged.

7. Overfitting and Underfitting

Overfitting happens when a model learns the training data too well, including its noise and random quirks, and then performs badly on new data. Its training score is high but its test score is much lower. Underfitting is the opposite: the model is too simple to capture the real pattern, so it performs poorly on both training and test data.

This is related to the bias-variance tradeoff. High bias means the model makes strong simplifying assumptions and underfits. High variance means the model is too sensitive to the particular training set and overfits. The goal is to find a balance with low total error.

Ways to reduce overfitting include collecting more training data, using a simpler model, applying regularization, using dropout in neural networks, stopping training early when validation error starts rising, and using cross-validation to detect the problem early.

Regularization adds a penalty for large weights. L1 regularization, used in Lasso regression, can shrink some weights exactly to zero, which performs feature selection. L2 regularization, used in Ridge regression, shrinks all weights smoothly toward zero. The strength of the penalty is a hyperparameter that must be tuned.

Hyperparameters are settings chosen before training, such as tree depth or learning rate. Grid search tries all combinations from a list, while random search samples combinations randomly. Both should be evaluated with cross-validation, never on the final test set.

8. Neural Networks and Deep Learning

An artificial neural network is made of layers of connected units called neurons. Each neuron multiplies its inputs by weights, adds a bias, and passes the result through an activation function. The network has an input layer, one or more hidden layers, and an output layer. Deep learning refers to networks with many hidden layers.

Activation functions add non-linearity so the network can learn complex patterns. ReLU, which outputs zero for negative inputs and the input itself for positive inputs, is the most common choice in hidden layers. Sigmoid is used for binary outputs, and softmax is used to turn the outputs of a multi-class problem into probabilities that add up to 1.

Training uses backpropagation. The network makes a prediction in a forward pass, the loss function measures the error, and then the error is sent backward through the network to compute how much each weight contributed. An optimizer such as Adam updates the weights to reduce the loss. This is repeated over many passes through the data, each called an epoch.

Convolutional Neural Networks (CNNs) are specialized for images. They use small filters that slide across the image to detect edges, textures, and shapes. Recurrent Neural Networks (RNNs) and LSTMs were designed for sequences such as text and time series. Today, Transformers have replaced RNNs for most language tasks.

Popular frameworks for building neural networks are PyTorch, TensorFlow, and Keras. Training large networks is much faster on a GPU, which can perform many calculations in parallel.

9. Natural Language Processing, Embeddings and RAG

Natural Language Processing (NLP) is the field of AI that deals with human language. Typical tasks include text classification, sentiment analysis, named entity recognition, machine translation, summarization, and question answering.

Computers cannot work directly with words, so text must be converted to numbers. Early methods such as bag-of-words and TF-IDF count how often words appear. A modern approach is the embedding: a list of numbers, called a vector, that represents the meaning of a word, sentence, or paragraph. Texts with similar meanings have embeddings that are close together in vector space.

Similarity between two embeddings is usually measured with cosine similarity, which compares the angle between the two vectors. A value close to 1 means very similar meaning, while a value near 0 means unrelated. The Sentence-Transformers library provides ready-made models that turn sentences into embeddings.

A Large Language Model (LLM) is a very large Transformer model trained on massive amounts of text so that it can understand and generate language. LLMs can hallucinate, which means they sometimes produce confident but incorrect statements, and they do not know private or very recent information.

Retrieval-Augmented Generation (RAG) reduces these problems. First, documents are split into chunks and each chunk is converted into an embedding and stored in a vector database such as FAISS or Chroma. When a user asks a question, the question is also embedded, and the most similar chunks are retrieved. These chunks are added to the prompt, and the LLM writes an answer based only on that retrieved context. Showing the source page or document for each answer builds user trust.

10. The ML Project Workflow and Deployment

A typical machine learning project follows these stages. First, define the problem and decide how success will be measured. Second, collect the data. Third, explore the data with charts and summary statistics, which is called exploratory data analysis (EDA). Fourth, clean and preprocess the data. Fifth, train several models and compare them using cross-validation. Sixth, tune the best model. Seventh, evaluate it once on the held-out test set. Eighth, deploy it and monitor it.

Deployment means making the model available to real users. A common approach is to save the trained model to a file and wrap it in a web API using a framework such as FastAPI or Flask. The API accepts input data, runs the model, and returns the prediction. For simple demos, Streamlit lets you build an interactive web app with only Python.

Docker packages the application with all its dependencies into a container, so it runs the same way on any machine. Version control with Git and GitHub keeps the code organized and lets recruiters see your work. A good README file explains the problem, the approach, the results, and how to run the project.

After deployment, models must be monitored. Data drift occurs when the incoming data changes over time and no longer matches the training data, which causes performance to drop. Teams track metrics, collect feedback, and retrain the model periodically. This practice is known as MLOps.

Tips for beginners: start with small projects, finish them end to end, write clear documentation, and always report results with numbers. A project that is deployed and explained well is more valuable than a complicated project that is never finished.